GRAPPA: Generalizing and Adapting Robot Policies via Online Agentic Guidance

Arthur Bucker[1], Pablo Ortega-Kral[1], Jonathan Francis[1,2], Jean Oh[1]
[1]Robotics Institute, Carnegie Mellon University
[2]Robot Learning Lab, Bosch Center for Artificial Intelligence
{abucker, portegak, jmf1, jeanoh}@cs.cmu.edu

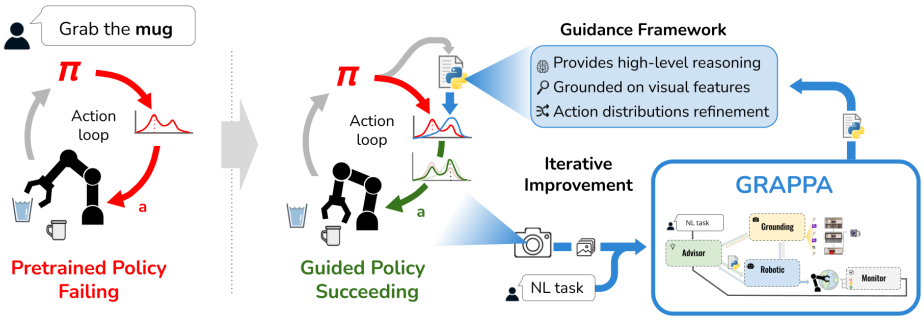

Fig. 1: An illustration of how GRAPPA intervenes in the action loop of pre-trained robotic policies in failure cases to provide visuomotor guidance generated with an agentic framework of agents to shift the action distribution for correct execution.

***Abstract*— Robot learning approaches such as behavior cloning and reinforcement learning have shown great promise in synthesizing robot skills from human demonstrations in specific environments. However, these approaches often struggle to generalize to unseen real-world settings because they rely on task-specific demonstrations or complex simulators. While foundation models (e.g., LLMs, VLMs) offer rich semantic understanding from internet-scale data, using this knowledge to enable robotic systems to understand the underlying dynamics of the world, generalize policies across different tasks, and adapt policies to new environments remains an open challenge. To alleviate these limitations, we propose an agentic framework for robot self-guidance and self-improvement that comprises a set of role-specialized conversational agents, including a high-level advisor, a grounding agent, a monitoring agent, and a robotic agent. Our framework iteratively grounds a base robot policy on relevant objects in the environment and uses visuomotor cues to shift the policy's action distribution toward more desirable states online, while remaining agnostic to the subjective configuration of a given robot hardware platform. We demonstrate that our approach can effectively guide manipulation policies to achieve significantly higher success rates, both in simulation and in real-world experiments, without the need for additional human demonstrations or extensive exploration. Code and videos are available at: `https://agenticrobots.github.io/`**

I. Introduction

In recent years, foundation models, including large language models (LLMs) and vision-language models (VLMs), have enabled impressive capabilities for understanding context, scenes, and dynamics in the world. Emergent capabilities, such as in-context learning, have shown great potential for transferring knowledge across domains, e.g., via few-shot demonstrations or zero-shot inference. However, the application of these models to robotics remains limited, given the intrinsic complexity and scarcity of human-robot interactions, as well as the lack of large-scale datasets of human-annotated data or demonstrations [1], [2].

Existing integrations of foundation models in robotics typically follow two paradigms. The first utilizes foundation models as zero-shot planners, code-generators, and task-descriptors—aiming to use the foundation model to provide some high-level instruction to a low-level policy or to describe the policy's actions in natural language [3], [4], [5], [6] or generated plans as code [7], [8]. While these methods demonstrate strong reasoning capabilities, they require learning additional mappings to interface the generated natural language instructions with the low-level policy or rely on pre-existing handcrafted primitives to compose. The secondary paradigm finetunes foundation models for robot learning, using large corpora of robotic demonstration datasets to supervise part or all of the policy to learn to perform tasks in an end-to-end manner, [9], [10]. While effective in-domain, methods often struggle to generalize to novel object categories, tasks, and environments unseen during training, and may suffer from negative transfer when trained on very diverse collections of robot data [11], [12].

In this paper, we extend the deployability of robot policies by using foundation models to generate low-level visuomotor guidance to handle out-of-distribution scenarios, such as sim-to-real differences or new tasks and robot platform variations (Figure 3). We design an agent-based framework that employs specialized conversational agents that collaborate to refine the action distribution of a base policy through grounded visuomotor guidance (Figure 1). The advisor agent orchestrates guidance code generation by querying the grounding agent to locate target objects, consulting the robotic agent to validate code feasibility for the robot platform, and incorporating feedback from the monitor

agent on rollout failures. The resulting guidance function evaluates candidate actions from the base policy by scoring predicted future states, producing a biasing distribution that shifts actions toward task completion.

In simulated and real-world experiments, we comprehensively and empirically demonstrate that our framework for **G**ene**r**alizing and **A**da**p**ting Robot **P**olicies via Online **A**gentic Guidance (GRAPPA) provides robustness for a variety of representative base policy classes to sim-to-real transfer paradigms and out-of-distribution settings, such as unseen objects.

A summary of our contributions is as follows:

- We propose GRAPPA, an agentic robot policy framework that self-improves by generating a guidance function to update base policies' action distributions, online. Our framework is capable of learning skills from scratch after deployment and generalizes across various base policy classes, tasks, and environments.
- For robustness against cluttered and unseen environments, we propose a grounding agent that performs multi-granular object search, enabling flexible visuomotor grounding.
- We provide experimental results showcasing GRAPPA as a helper tool for aiding in the sim-to-real transfer of policies without the need for extensive data collection.

II. Related Work

Vision-Language Models for Robot Learning: Several works explore the notion of leveraging pre-trained or fine-tuned Large Language Models (LLMs) and/or Vision-Language Models (VLMs) for high-level reasoning and planning in robotics tasks [1], [5], [7], [13], [8], [14], [15], [16], [17], [18], [19], [20]—typically decomposing high-level task specification into a series of smaller steps or action primitives, using system prompts or in-context examples to enable powerful chain-of-thought reasoning techniques. This strategy of encouraging models to reason in a step-wise manner before outputting a final answer has led to significant performance improvements across several tasks and benchmarks [1]. Despite these promising achievements, these approaches rely on handcrafted primitives [5], [13], [7], [17], struggle with low-level control, or require large datasets for retraining. Furthermore, various approaches that leverage VLMs for robot learning suffer from a granularity problem when using off-the-shelf models in a single-step/zero-shot manner [21] or are unable to perform failure correction without costly human intervention [13], [17], [22], [21]. In contrast, our framework bridges high-level reasoning with low-level control, by leveraging an agentic framework for online modification of a base policy's action distribution at test-time, without requiring human feedback or datasets for fine-tuning. Moreover, we mitigate the granularity problem by proposing a flexible and recursive grounding mechanism that uses VLMs to query open-vocabulary perception models.

Agent-based VLM frameworks in Robotics: Rather than using single VLMs in an end-to-end fashion, which might incur issues in generalization and robustness, various works have sought to orchestrate multiple VLM-based agents to work together in an interconnection multi-agent framework. Here, multiple agents can converse and collaborate to perform tasks, yielding improvements for the overall framework in online adaptability, cross-task generalization, and self-supervision [23], [24], [25]. These *agentic* frameworks have provided possibilities for enabling the identification of issues in task execution, providing feedback about possible improvements. Challenges remain, however, in that this feedback is often not sufficiently grounded on the spatial, visual, and dynamical properties of embodied interaction to be useful for policy adaptation; instead, the generated feedback is often too high-level or provides merely binary signals of success or failure.

Self-guided Robot Failure Recovery: [26] offer an analysis of frameworks for leveraging VLMs as behavior critics. Some approaches have explored integrating such pre-trained models to improve the performance of reinforcement learning (RL) algorithms. For instance, [18] use LLMs in a zero-shot fashion to design and improve reward functions, however this approach relies on human feedback to generate progressively human-aligned reward functions and further requires simulated retraining via RL, with high sample-complexity. On the other hand, [27] avoid the need for explicit human feedback by directly using a VLM (CLIP) to compute the rewards to measure the proximity of a state (image) to a goal (text description), enabling gains in sample-efficiency for guiding a humanoid robot to perform various maneuvers in the MuJoCo simulator. A limitation of this approach, however, lies in the difficulty of generating rewards for long-horizon or multi-step tasks, which are characteristic of tasks involving complex agent-object interactions. [6] present a framework for detecting and analyzing failed executions automatically. However, their system focuses on explaining failure causes and proposing suggestions for remediation, as opposed to also performing policy correction. In this paper, we propose a framework that directly adapts a base policy's action distribution, during deployment, without requiring additional human feedback.

III. Grounding Robot Policies with Guidance

A. Problem Formulation

We consider a pre-trained stochastic policy $\pi : O \times S \to A$ that maps observations o_t and robotic states s_t to action distributions $a_{\pi,t}$, at each time step t. Our objective is to generate a guidance distribution Specifically, we aim to develop a modified policy $\pi_{\text{guided}} : O \times S \to A$ that achieves better performance on tasks where the original policy π struggles. We define this new policy as follows:

$$\pi_{guided}(a_{g,t}|o_t, s_t) = \pi(a_{\pi,t}|o_t, s_t) * G(a_{\pi,t}|o_t, s'_{t+1}), \quad (1)$$

where $a_{g,t}$ denotes the guided action sampled from the modified policy π_{guided}, and $G : A \times O \times S \to [0, 1]$ is a guidance function that evaluates each candidate action $a_{\pi,t}$ by considering the observation o_t and the predicted future state s'_{t+1}, producing a guidance score in $[0, 1]$. By applying G to all actions in the distribution from π and normalizing the resulting scores (as shown in Algorithm 1),

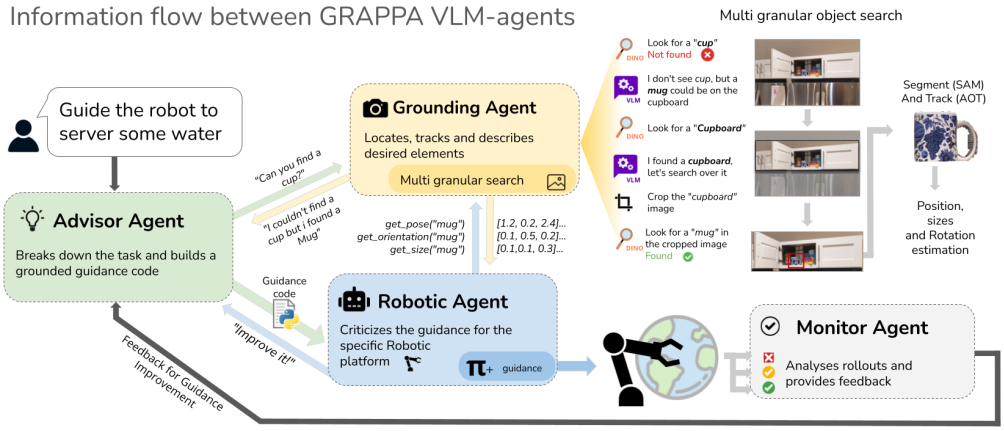

Fig. 2: Information flow for guidance code generation. **a)** The advisor agent orchestrates generation, calling other agent's and refining code based on their feedback. **b)** The grounding agent locates target objects specified by the advisor using segmentation and classification models. **c)** The robotic agent uses a Python interpreter to judge the adequacy of the code. **d)** The monitor agent analyses the sequence of frames corresponding to the rollout of the guidance and give feedback on potential improvements.

we obtain a proper probability distribution—the guidance distribution—that reshapes the base policy's action probabilities.The '$*$' operator here denotes the operation of combining both distributions conceptually, which we explore in detail in Section III-D. For the scope of this project, we assume that a dynamics model $\mathcal{D} : S \times A \to S$ is available, which can forecast possible future states of the robot $s'_{t+1} = \mathcal{D}(s_t, a_{\pi,t})$ given the current state s_t and action $a_{\pi,t}$.

Focusing on leveraging the world knowledge of Vision Language Models, while avoiding adding latency to the action loop, we choose to express these guidance functions as Python code. By integrating these code snippets into action loop of the base policies, we eliminate the need of time-consuming queries to large reasoning models.

B. A Multi-agent Guidance framework for Self Improvement

To generate the guidance function G, we employ a multi-agent framework consisting of four specialised conversational agents, equipped with visual grounding and tool-use capabilities. System prompts for each agent can be found on the supplementary website.

Advisor Agent: A Vision Language Model is responsible for breaking down the task and communicating with the other agents to generate a sound guidance function for a given task.

Grounding Agent: A Vision Language Model that performs multi-granular object search by iteratively querying open-vocabulary detection and segmentation models to locate [28], [29], track [30], and describe elements relevant to the task execution. Unlike prior work that uses VLMs for grounding in a single-step or zero-shot manner [31], [21], our grounding agent employs a recursive search strategy that dynamically adjusts the image search space and refines the natural language queries based on detection success and a self-reflection loop. These design choices enable better use of off-the-shelf detection and segmentation models for more robust visual grounding, as shown in the Appendix section of the project website.

Monitor Agent: Responsible for identifying the causes of the failures in the unsuccessful rollouts, the Monitor Agent consists of a Vision Language model equipped with a key frame extractor.

Robotic Agent: Language Model equipped with descriptions of the robot platform, a robot's dynamics model and wrapper functions for integration with the base policy. It criticizes the provided guidance functions to reinforce its relevance to the task and alignment with the robot's capabilities.

C. Guidance Procedure

The agents interact through natural language and query their tools to iteratively produce guidance code tailored to the task, environment, and robot capabilities 2. Given a task expressed in natural language and an image of the initial state of the environment, the *Advisor Agent* uses Chain-of-Tought [32] strategy to generate a high-level plan to accomplish the task. By querying a *Grounding Agent* and the *Robotic Agent*, the Advisor can collect relevant information about trackable objects and elements in the environment, as well as the capabilities and limitations of the robotic platform.

The Grounding Agent uses Grounding Dino [29] and the Segment Anything Model (SAM) [28] to locate the elements across multiple granularities and levels of abstraction. If an

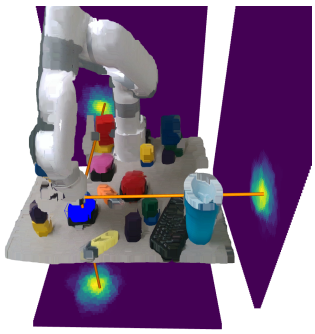

Fig. 3: Heatmap visualization of the guidance distribution, generated online by our proposed agentic framework. The distribution expresses the relevance of each possible future state for completing the task (e.g. "press the blue button"). The guidance is then used to bias the robot policy's action distribution towards the desirable behavior.

object (e.g., "cup") is not found, it searches for semantically similar items (e.g., "mug") or parent objects (e.g., "shelf") to refine detection. Found objects are tracked via DeAOT [33], enabling a flexible segment-and-track pipeline [30]. Object statuses inform the Advisor Agent when to revise a plan or generate guidance.

The Robotic agent acts as a critic to improve the guidance function generated. Equipped with a Python interpreter and details of the base policy's action space, the agent can evaluate the guidance function for feasibility and relevance to the robot's capabilities. Once a function meets the system's requirements, it is saved for use in the action loop, along with the dynamics model, to provide a guidance score for possible actions sampled from the base policy.

After a rollout is executed, and if task completion fails, the Monitor Agent is triggered to analyze the causes of the failure. By extracting key frames from the rollout video using PCA [34] and K-means clustering, the agent can feed a relevant and diverse set of images to the Visual Language Model, prompted to access the failure causes. In the iterative applications of our framework, the Monitor Agent provides this feedback to the Advisor Agent, which can use it to refine the guidance functions generated in previous iterations.

Temporal-aware Guidance Functions. Inspired by recurrent architectures, we instruct the agents to generate a guidance function conditioned on a customizable hidden state (h_t) expressed as an optional dictionary parameter, as shown in the following example:

```
# Guidance function example in the context of
    grabbing a mug
def guidance_code(state,
    hidden_state={"mug_reached": False,"mug_grabbed
        ":False}):
    #available grounding functions
    #x,y,z = get_pose("mug")
    #h,w,d = get_size("mug")
    #rx,ry,rz = get_orientation("mug")
    ...
    return score, new_hidden_state
```

The idea of using abstraction in a hidden state has proven to significantly improve guidance performance, allowing guidance functions to track task progress and adapt to longer-horizon tasks. The guided policy can thus be written as:

$$\pi_{guided}(a_{g,t}|o_t, s_t, h_t) = \pi(a_{\pi,t}|o_t, s_t) * G(a_{\pi,t}|o_t, s'_{t+1}, h_t). \quad (2)$$

The complete guidance procedure is summarized in Algorithm 1. Note that we refer to the self-orchestrated conversation between the agents that yields the guidance code as the function `Generate_Guidance_Function`.

D. Guidance and policy integration

Our framework supports both continuous and discrete action spaces. In this section, we discuss combining guidance functions with base policy action distributions and adapting deterministic regression models.

Action-space Adaptation. We assume access to a dynamics model $\mathcal{D}$ that forecasts future robot states given a possible action $a'_{\pi,t}$. In the manipulation domain, this is often available as a forward kinematics model, a learned dynamics model, or a simulator. Oftentimes, the action space A of policies them-self is the same as the robot's state S, either being the joint angles of the robot or the gripper's end-effector pose. For the last cases, where both the action and state space are expressed in $SE(3)$, integrating the guidance function with a base policy would only require a multiplication of the guidance scores with the action probabilities of the base policy. In other scenarios, adapting the robot's action and state space to match the representation of the visual cues (position, orientation, and size) would be required.

Algorithm 1 Guidance procedure of `GRAPPA`

Input
π: Base policy
$\mathcal{D}$: Dynamics Model
env: Environment

for each episode **do**
  $o_t,\ s_t \leftarrow$ env.init ▷ *observation and initial state*
  $G \leftarrow$ Generate_Guidance_Function($o_t,\ s_t$)
  $h_t \leftarrow$ Get_Hidden_State($G(o_t, s_t)$)
  for each time step t **do**
    $\mathcal{A}_{\pi,t} \leftarrow \{\pi(o_t, s_t)^i\}_{i=0}^{n}$ ▷ *Sample n actions*
    $\pi_t \leftarrow \pi(\mathcal{A}_{\pi,t}|o_t, s_t)$ ▷ *Get action probabilities*
    $\mathcal{S}_{\pi,t} \leftarrow \mathcal{D}(s_t, \mathcal{A}_{\pi,t})$ ▷ *Infer possible future states*
    $G_{\pi,t} \leftarrow G(o_t, \mathcal{S}_{\pi,t}, h_t)$ ▷ *Compute the guidance for the sampled possible future states*
    $G_{\pi,t} \leftarrow G_{\pi,t}/|G_{\pi,t}|$ ▷ *Normalize guidance score forming a guidance distribution*
    $\pi_{\text{guided},t} \leftarrow \pi_t * G_{\pi,t}$ ▷ *Combine distributions*
    $a_t \leftarrow \mathcal{A}_{\pi,t}\,[\text{argmax}(\pi_{\text{guided},t})]$ ▷ *Select the best a*
    $o_t,\ s_t \leftarrow$ env.step(a_t) ▷ *Execute a_t, update state s_{t+1} and observation o_{t+1}*
    $h_t \leftarrow$ Get_Hidden_State($G(o_t, s_t, h_t)$) ▷ *Update*

Considering the visual grounding, the action space and the state space share the same representation ($SE(3)$), the operation to combine the guidance function with the base policy can be expressed as an element-wise weighted average

$$\pi_{guided} = (1 - \alpha)\pi + \alpha G, \quad (3)$$

where $\alpha \in [0, 1]$ represents the percentage of guidance applied with respect to the base-policies distribution and is here denoted as *guidance factor*. This equation represents an element-wise weighted sum of the two distributions, where $\alpha = 0$ yields full reliance on the base policy π, while $\alpha = 1$ yields full reliance on the guidance function G.

Adaptation of Regression Policies. To properly leverage the high-level guidance expressed in the guidance functions and the low-level capabilities of the base policy, it is desired that the policy's action space be expressed as a distribution. In the case of regression policies that do not provide uncertainty estimates, several strategies can be employed to infer the action distribution. One common approach is to assume a Gaussian distribution centered at the predicted value and compute the variance using ensembles of models trained with

TABLE I: Performance improvement on the RL-Bench [35] benchmark, by applying 5 iterations of guidance improvement over unsuccessful rollouts.

Model	turn tap	open drawer	sweep to dust-pan of size	meat off grill	slide block to color target	push buttons	reach and drag	close jar	put item in drawer	stack blocks	**Avg. Success**
Act3D 25 demos/task	76	76	96	64	92	84	96	48	60	0	69.2
+1% guidance	80 (+4)	96 (+20)	96	84 (+20)	92	84	100 (+4)	84 (+36)	80 (+20)	8 (+8)	**80.4 (+11.2)**
+10% guidance	88 (+12)	100 (+24)	96	88 (+24)	92	84	100 (+4)	60 (+12)	80 (+20)	0	78.8 (+9.6)
Act3D 10 demos/task	32	60	84	16	60	72	68	32	44	8	47.6
+1% guidance	44 (+12)	88 (+28)	88 (+4)	24 (+8)	68 (+8)	72	76 (+8)	52 (+20)	60 (+16)	8	**58 (+10.4)**
+10% guidance	44 (+12)	64 (+4)	84	20 (+4)	68 (+8)	76 (+4)	76 (+8)	40 (+8)	56 (+12)	8	53.6 (+6)
Act3D 5 demos/task	24	0	84	4	8	32	8	8	12	0	18
+1% guidance	48 (+24)	16 (+16)	84	8 (+4)	12 (+4)	40 (+8)	24 (+16)	20 (+12)	20 (+8)	0	**27.2 (+9.2)**
+10% guidance	24	0	84	8 (+4)	12 (+4)	44 (+12)	20 (+12)	8	20 (+8)	0	22 (+4)
Diffuser actor 5 demos/ task	24	64	40	28	44	68	40	24	44	0	37.6
+1% guidance	40 (+16)	92 (+28)	64 (+24)	40 (+12)	44	68	52 (+12)	24	84 (+40)	0	**50.8 (+13.2)**
+10% guidance	40 (+16)	84 (+20)	52 (+12)	28	52 (+8)	68	48 (+8)	32 (+8)	76 (+32)	0	48 (+10.4)

different initialization, different data samples, or different dropout seeds or different checkpoint stages [36]. Other strategies to infer the distributions of the model include using bootstrapping, Bayesian neural networks, or using Mixture of Gaussians [37].

IV. Experiments & Results

A. Experimental Setup

We demonstrate GRAPPA's efficacy in simulation on the RL-Bench benchmark [35] and on two real-world tasks. The real-world setup uses a UFACTORY Lite 6 robot arm with UFACTORY Gripper Lite (parallel jaw gripper) mounted on a workbench, and an Intel RealSense D435i RGB-D camera for perception. Experiments ran on a desktop with an NVIDIA RTX 4080 GPU, 64GB RAM, and AMD Ryzen 7 8700G CPU.

Base Policies: We evaluate the effectiveness of our guidance framework using different base policies, *Act3D* [38], *3D Diffuser Actor* [39], and a RandomPolicy. All policies plan in a continuous space of translations, rotations, and gripper state ($SE(3) \times \mathbb{R}$), however they utilize different inference strategies:

- ***Act3D*** samples waypoints in $\mathbb{R}^3$ and predicts orientation and gripper state for the best waypoint, combining classification and regression.
- ***3D Diffuser Actor*** uses diffusion to compute target waypoints and infers orientation and gripper state from a single forecast, tackling this as a regression task.
- ***RandomPolicy*** denotes either framework with randomly initialized weights (untrained for the specific task).

These fundamentally different policy outputs make them ideal for testing our framework. Furthermore, their common action and state spaces representation ($SE(3) \times \mathbb{R}$) enables straightforward integration with our grounding models.

As described in Section III-D, we adapt the regression components to transform single predictions into action space distributions. We assume a Gaussian distribution with mean at predicted values and fixed standard deviation. Act3D's classification output (waypoint positions) was directly treated as samples from a distribution over $\mathbb{R}^3$.

The integration of base policies with the guidance distributions was performed by applying a weighted average parameterized by α as shown in Equation 3.

B. Experimental Evaluation

Our experimental evaluation aims to address the following questions: (1) Does GRAPPA enable policies to bridge the sim-to-real gap for deployment on real-world tasks and in out-of-distribution settings? (2) Does GRAPPA improve the performance of pre-trained base policies on specific robotics tasks and environments without additional human demonstrations? (3) Does GRAPPA enable policies to learn new skills from scratch? (4) What is the effect of guidance on expert versus untrained policies?

Does GRAPPA enable policies to bridge the sim-to-real gap for deployment on real-world tasks and in out-of-distribution settings? We train a *3D Diffuser Actor* policy on 10 tasks with 100 demonstrations each using a single RGB-D camera (*GNFactor* [40] setting), exclusively from simulation. This policy showed stronger overall simulation performance. We deploy it in real-world button-pressing in a cluttered environment (Figure 3). To facilitate sim-to-real transfer, we matched real-world workspace dimensions, scale, and alignment to simulation data, with similar camera placement.

TABLE II: Real-world performance improvement in a sim-to-real setting. *Diffuser actor* policy trained on 10 tasks in simulation (*GNFactor* [40] setup: 100 demos/task) and evaluated on the tasks *"Push buttons"* with different guidance levels. **Red** cells refer to failures due to timeouts in task execution. **Orange** cells refer to errors in perception. **Green** cells refer to successes.

Policy	Task completion Success Rate [%]	Error Breakdown
Naive Sim2Real	0.0	
+15% guidance	66.7	
+50% guidance	50.0	
+75% guidance	100	
Finetuned Baseline *(10 real-world demos)*	33.0	
+15% guidance	50.0	
+50% guidance	16.7	
+75% guidance	100	

As depicted by Table II, the cluttered scenario in Figure 3 has shown to be out-of-distribution scenarios for the base policy that was solely trained in simulation (Naive Sim2Real). Fine-tuning the policy with 10 real-world demonstrations of pressing buttons in uncluttered environments has proven to slightly increase the performance of the policy. These base policies, combined with our guidance framework, achieve a higher success rates in 5 of 6 scenarios with

guidance. In the only exception case (finetuned baseline + 50%), the only exception (finetuned baseline + 50%) occurred when perception models failed to detect and track the target button, leading to detrimental guidance.

A known limitation of diffusion-based policies is their struggle to generalize beyond training data. To test GRAPPA's ability to address this, we take a simulation-trained policy and fine-tune on 15 real-world pick-and-place demonstrations (Figure 4). We assess two generalization axes: 1) position generalization with same objects at different positions; and 2) appearance generalization with varied object types but same semantic class. Table III shows results for 10 rollouts: 3D Actor Diffuser fails all trials for both generalization types, while GRAPPA (50% guidance) succeeds in 3/5 cases for position and 4/5 for appearance generalization.

TABLE III: GRAPPA guiding the base policy for out-of-distribution cases, illustrated in Figure 4.

Baseline	Position SR	Appearance 1	2	3	4	5	SR
3D Actor Diffuser	0/5	✗	✗	✗	✗	✗	0/5
+ GRAPPA (50%)	3/5	✓	✗	✓	✓	✓	4/5

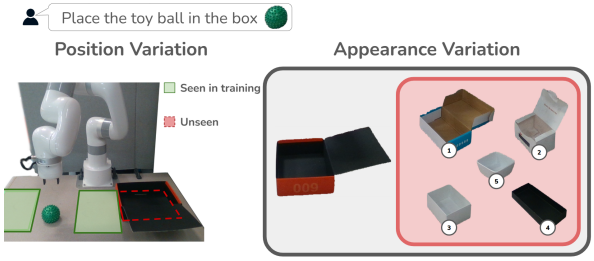

Fig. 4: GRAPPA guiding the base policy for out-of-distribution cases. The task involves grasping a deformable toy ball and placing it inside a box.

Across both real-world tasks, a higher percentage of guidance is necessary compared to the simulation results seen in Table I. This hints at two practical properties of GRAPPA's guidance effect. First, there must be a trade-off between high-level guidance (provided by GRAPPA) and low-level control (provided by the base policy); in more complex tasks, elevated guidance values disrupt the robot's low-level motions, leading to some failure cases. We compare trajectories of the failing base policy versus GRAPPA-guided in Figure 5. Second, the optimal guidance level is also dependent on the performance of the base policy itself. Notably, while the simulation-based policy has already achieved a good performance on the test set, it is unable to deal with out-of-distribution cases and requires much more correction in the real-world setting.

Does GRAPPA improve the performance of pre-trained base policies on specific robotics tasks and environments without additional human demonstrations? We first assess the effect of the proposed guidance on the *Act3D* and *3D Diffuser Actor* baselines following the *GNFactor* [40] setup(single RGB-D camera, table-top manipulation, 10 challenging tasks with 25 variations each). Guidance is iteratively generated for the failure cases. For the failed rollouts, our policy improvement framework ran for 5 iterations. As shown in Table I, the framework was able to improve the success rate of the base policy on most of the tasks, with the best results achieved by using 1% guidance. The low amount of guidance has shown to be enough to bend the action distribution to the desired direction, while still preserving the low-level nuances captured by the base policy. This suggests that GRAPPA is capable of improving base policies by adding abstract understanding and grounding of the desired task, while preserving the low-level movement profiles captured by the original policies.

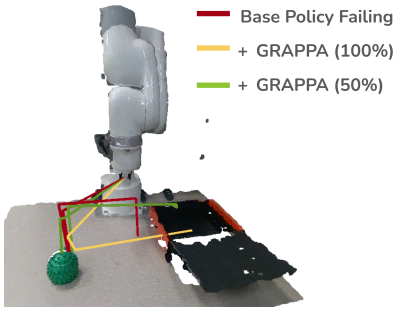

Fig. 5: Effect of guidance percentages on a base policy failure. **Red** shows the base policy failing in an out-of-distribution scenario; with 100% of guidance (**yellow**), the end position is correct, but it has *lost low-level notions*. By balancing both with intermediate guidance (50%) shown in **green**, we can complete the task.

Does GRAPPA enable policies to learn new skills from scratch? We evaluated the performance of the framework on learning new skills from scratch on 4 tasks of the RLBench benchmark: *turn tap*, *push buttons*, *slide block to color target*, *reach and drag*. In this setup, we initialized the Act3D policy with random weights and applied 100% of guidance ($\alpha = 1.0$) over the policy for the x, y, and z components. Only leveraging the waypoint sampling mechanism of Act3D and overwriting its distribution with the values queried from the guidance functions generated. The results show that the framework is capable of learning new skills from scratch, achieving a higher success rate than the base policy pre-trained on 5 demonstrations/tasks for the tasks "turn tap" and "push buttons" tasks. When utilizing only the untrained Act3D policy (without guidance) the policy achieved 0 success rate on the tasks. Figure 6 demonstrates the iterative improvement of our guidance framework, wherein the guidance code generated for each failed rollouts from the previous iteration is iteratively updated.

As shown in Figure 6, the success rate plateaus after several iterations. This limitation stems from two primary factors: (1) persistent perception failures, where false positive detections consistently misguide the policy, and (2) the monitor agent's inability to identify missing low-level behaviors necessary for task completion. Since GRAPPA reweights actions from the base policy's sampling rather

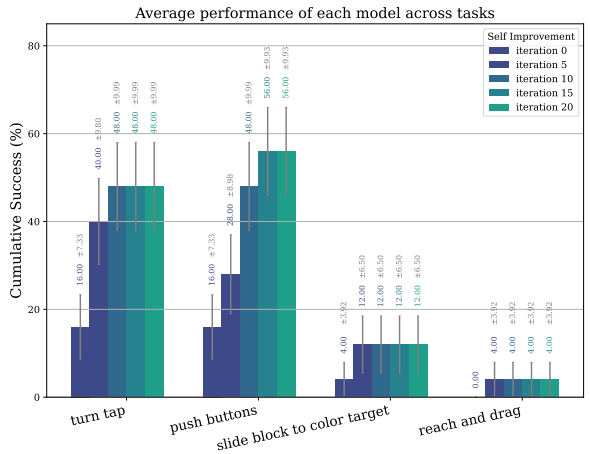

Fig. 6: Performance of our framework on learning new skills from scratch (guidance over untrained policies), and iteratively improving the guidance functions generated.

than generating entirely new strategies, it cannot overcome failures beyond the base policy's action sampling capabilities or correct errors introduced by unreliable perception. In the project website, we present an analysis of the remaining failed rollouts, highlighting these limitations and modes of failure.Notably, several variations of "turn tap" and "reach and drag" requiring precise orientation control were solved by guiding only Cartesian components. Qualitative analysis shows these variations succeeded by tapping the end-effector on target objects.

What is the effect of guidance on expert versus untrained policies? As discussed previously, GRAPPA can learn tasks from scratch using a random base policy with 100% of guidance (α). Since the policy remains unaffected, the iterative learning produces guidance scripts that capture high-level task understanding (spatial relationships, completion criteria, etc.). The learning process is visible by comparing guidance scripts across iterations in the project website.

On the other hand, applying the guidance to an existing expert policy aims to shift the action distribution to account for failure cases, like potentially out-of-distribution scenes. The goal here is to use the gained high-level understanding to aid the policy in task completion. Table I shows that adding too much weight to the guidance function can yield diminishing returns as it can overpower the nuanced low-level details from the expert policy: notice that the performance gain across the board is bigger using 1% of guidance.

V. Discussion

Guidance factor selection: The guidance factor α in Equation 3 balances the influence of the base policy and the guidance function. The discrepancy in optimal guidance ratios between simulation (Table I, $\alpha = 0.01$) and real-world experiments (Table II, $\alpha = 0.75$) reflects two factors: (1) the trade-off between high-level guidance and low-level control—simulation policies perform well in-distribution and require minimal correction, while real-world settings present out-of-distribution challenges (clutter, sim-to-real gap) demanding more guidance; (2) optimal guidance depends on deployment settings (robot, policy quality, action space)—near-expert simulation policies need subtle adjustments, while real-world deployment requires higher ratios to bridge the domain gap.

In our experiments, α was selected heuristically via grid search over $\{0.01, 0.1, 0.25, 0.5, 0.75\}$ based on validation performance. Future work could automate this selection through: (1) **Distributional distance metrics:** For properly calibrated base policies, comparing the distance between the base policy and guidance distributions using empirical measures (e.g., KL divergence, Wasserstein distance) could inform a principled selection of α. However, non calibrated base-policies can present arbitrarily biased action distribution, making this apporach unrealistic for most out-of-distribution cases. (2) **Monitor-driven adaptation:** The monitor agent could dynamically adjust α based on observed task progress and failure patterns, increasing guidance when the policy struggles and decreasing it when the base policy performs well. (3) **Domain similarity assessment:** Automatically comparing the similarity between the test setting and the training dataset (e.g., using visual feature distances or task complexity metrics) could provide an initial estimate of the required guidance level, with higher similarity suggesting lower α values. Notably, these approaches could enable dynamic guidance factors that adapt throughout rollout execution—increasing α during challenging task phases (e.g., object manipulation in clutter) and decreasing it when the base policy operates within its competence region, thereby optimizing the balance between autonomous behavior and corrective guidance at each timestep.

VI. Conclusion

Summary: In this work, we proposed GRAPPA, a novel framework for the self-improvement of embodied policies. Our self-guidance approach leverages the world knowledge of a group of conversational agents and grounding models to guide policies during deployment. We demonstrated the effectiveness of our approach in autonomously improving manipulation policies and learning new skills from scratch, in simulated RL-bench benchmark tasks and in two challenging real-world tasks. Our results show that the proposed framework is especially effective in improving the following high-level task structures and key steps to solve the task. This capability can be well suited for improving pre-trained policies that struggle with long-horizon tasks or for learning new simple skills from scratch.

Limitations: From an analysis of the guided rollouts, a few of the task variations proved challenging for the perception models used by the grounding agent, leading to false positive detections or failure to locate specific objects. This was mainly observed in the simulation, given that simulated graphics are often not represented in the training set of detection models. A potential solution is to integrate more robust object detection models or verification procedures to ensure the correct detection of objects in the scene. Additionally, occasional scene understanding errors by the VLM led to inaccurate guidance and unexpected behaviors.

References

[1] Y. Hu, Q. Xie, V. Jain, J. Francis, J. Patrikar, N. Keetha, S. Kim, Y. Xie, T. Zhang, Z. Zhao, *et al.*, "Toward general-purpose robots via foundation models: A survey and meta-analysis," *arXiv preprint arXiv:2312.08782*, 2023.

[2] S. Yenamandra, A. Ramachandran, M. Khanna, K. Yadav, J. Vakil, A. Melnik, M. Büttner, L. Harz, L. Brown, G. C. Nandi, *et al.*, "Towards open-world mobile manipulation in homes: Lessons from the neurips 2023 homerobot open vocabulary mobile manipulation challenge," *arXiv preprint arXiv:2407.06939*, 2024.

[3] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, "Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning," in *7th Annual Conference on Robot Learning*, 2023.

[4] S. Saxena, B. Buchanan, C. Paxton, B. Chen, N. Vaskevicius, L. Palmieri, J. Francis, and O. Kroemer, "Grapheqa: Using 3d semantic scene graphs for real-time embodied question answering," *arXiv preprint arXiv:2412.14480*, 2024.

[5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[6] Z. Liu, A. Bahety, and S. Song, "Reflect: Summarizing robot experiences for failure explanation and correction," *arXiv preprint arXiv:2306.15724*, 2023.

[7] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9493–9500.

[8] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "Progprompt: Generating situated robot task plans using large language models," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 11 523–11 530.

[9] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, *et al.*, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024.

[10] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[11] A. Padalkar, A. Pooley, A. Jain, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Singh, A. Brohan, *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models," *arXiv preprint arXiv:2310.08864*, 2023.

[12] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.

[13] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, *et al.*, "Inner monologue: Embodied reasoning through planning with language models," *arXiv preprint arXiv:2207.05608*, 2022.

[14] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 10 608–10 615.

[15] H. Ha, P. Florence, and S. Song, "Scaling up and distilling down: Language-guided robot skill acquisition," in *Conference on Robot Learning*. PMLR, 2023, pp. 3766–3777.

[16] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, "Task and motion planning with large language models for object rearrangement," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 2086–2092.

[17] Z. Michał, C. William, P. Karl, M. Oier, F. Chelsea, and L. Sergey, "Robotic control via embodied chain-of-thought reasoning," *arXiv preprint arXiv:2407.08693*, 2024.

[18] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," *arXiv preprint arXiv:2310.12931*, 2023.

[19] X. Li, M. Zhang, Y. Geng, H. Geng, Y. Long, Y. Shen, R. Zhang, J. Liu, and H. Dong, "Manipllm: Embodied multimodal large language model for object-centric robotic manipulation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18 061–18 070.

[20] Y. Mu, Q. Zhang, M. Hu, W. Wang, M. Ding, J. Jin, B. Wang, J. Dai, Y. Qiao, and P. Luo, "Embodiedgpt: Vision-language pre-training via embodied chain of thought," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[21] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "Voxposer: Composable 3d value maps for robotic manipulation with language models," *arXiv preprint arXiv:2307.05973*, 2023.

[22] J. Liang, F. Xia, W. Yu, A. Zeng, M. G. Arenas, M. Attarian, M. Bauza, M. Bennice, A. Bewley, A. Dostmohamed, *et al.*, "Learning to learn faster from human feedback with language model predictive control," *arXiv preprint arXiv:2402.11450*, 2024.

[23] M. Xu, P. Huang, W. Yu, S. Liu, X. Zhang, Y. Niu, T. Zhang, F. Xia, J. Tan, and D. Zhao, "Creative robot tool use with large language models," *arXiv preprint arXiv:2310.13065*, 2023.

[24] C. Zhang, X. Meng, D. Qi, and G. S. Chirikjian, "Rail: Robot affordance imagination with large language models," *arXiv preprint arXiv:2403.19369*, 2024.

[25] M. Parakh, A. Fong, A. Simeonov, T. Chen, A. Gupta, and P. Agrawal, "Lifelong robot learning with human assisted language planners," *arXiv e-prints*, pp. arXiv–2309, 2023.

[26] L. Guan, Y. Zhou, D. Liu, Y. Zha, H. B. Amor, and S. Kambhampati, "" task success" is not enough: Investigating the use of video-language models as behavior critics for catching undesirable agent behaviors," *arXiv preprint arXiv:2402.04210*, 2024.

[27] J. Rocamonde, V. Montesinos, E. Nava, E. Perez, and D. Lindner, "Vision-language models are zero-shot reward models for reinforcement learning," *arXiv preprint arXiv:2310.12921*, 2023.

[28] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment anything," *arXiv:2304.02643*, 2023.

[29] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, *et al.*, "Grounding dino: Marrying dino with grounded pre-training for open-set object detection," *arXiv preprint arXiv:2303.05499*, 2023.

[30] Y. Cheng, L. Li, Y. Xu, X. Li, Z. Yang, W. Wang, and Y. Yang, "Segment and track anything," 2023. [Online]. Available: https://arxiv.org/abs/2305.06558

[31] J. Gao, B. Sarkar, F. Xia, T. Xiao, J. Wu, B. Ichter, A. Majumdar, and D. Sadigh, "Physically grounded vision-language models for robotic manipulation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 12 462–12 469.

[32] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[33] Z. Yang and Y. Yang, "Decoupling features in hierarchical propagation for video object segmentation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[34] A. Maćkiewicz and W. Ratajczak, "Principal components analysis (pca)," *Computers & Geosciences*, vol. 19, no. 3, pp. 303–342, 1993.

[35] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "Rlbench: The robot learning benchmark & learning environment," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.

[36] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya, *et al.*, "A review of uncertainty quantification in deep learning: Techniques, applications and challenges," *Information fusion*, vol. 76, pp. 243–297, 2021.

[37] J. Mena, O. Pujol, and J. Vitrià, "A survey on uncertainty estimation in deep learning classification systems from a bayesian perspective," *ACM Computing Surveys (CSUR)*, vol. 54, no. 9, pp. 1–35, 2021.

[38] T. Gervet, Z. Xian, N. Gkanatsios, and K. Fragkiadaki, "Act3d: 3d feature field transformers for multi-task robotic manipulation," in *7th Annual Conference on Robot Learning*, 2023.

[39] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki, "3d diffuser actor: Policy diffusion with 3d scene representations," *arXiv preprint arXiv:2402.10885*, 2024.

[40] Y. Ze, G. Yan, Y.-H. Wu, A. Macaluso, Y. Ge, J. Ye, N. Hansen, L. E. Li, and X. Wang, "Gnfactor: Multi-task real robot learning with generalizable neural feature fields," 2024. [Online]. Available: https://arxiv.org/abs/2308.16891